

INGENIERÍA DE SISTEMAS

Tecnología Web II

ACTIVIDAD ACADÉMICA N°3

VisualSync: frontend en React e integración con la API REST

AUTORES

David Miguel Flores Medrano

Docente: Vladimir Wilmar Rojas Condori

Cochabamba-Bolivia

18-10-2026

Índice de Contenido

1.Configuración del proyecto frontend 1

2. Estructura de componentes 2

3. Interfaces principales 3

4. Gestión de estados 6

5. Navegación 7

6. Formularios 8

7. Validación..... 9

8. Consumo de API REST..... 10

9. Operaciones CRUD desde el frontend..... 11

10. Diseño responsive..... 12

11. Integración frontend-backend..... 13

Flujo completo de punta a punta 14

12. Manejo de errores 14

13. Pruebas funcionales..... 15

14. Repositorio actualizado 17

15.Anexos 18

Índice de Ilustraciones

Ilustración 1:Catálogo de servicios..... 5

Ilustración 2: Registro 5

Ilustración 3: Inicio de sesión 5

Ilustración 4: Mis reservas 6

Ilustración 5: Diseño responsive 13

Ilustración 6: Commits..... 18

Ilustración 7: Ver el catálogo sin iniciar sesión..... 18

Ilustración 8:Registro con datos inválidos 19

Ilustración 9: Registro con correo repetido..... 19

Ilustración 10: Inicio de sesión con clave incorrecta 20

Ilustración 11: Inicio de sesión correcto 20

Ilustración 12: Admin crea, edita y desactiva un paquete 20

Ilustración 13: Cliente solicita una reserva..... 20

Ilustración 14: Reservar una fecha ocupada..... 21

Ilustración 15: Subir comprobante válido 21

Ilustración 16: Admin rechaza el pago con motivo..... 21

Ilustración 17: Admin aprueba el pago..... 22

Ilustración 18: Galería con pago pendiente..... 22

Ilustración 19: Galería con pago aprobado 22

Ilustración 20: Vista en móvil, tableta y escritorio..... 23

1.Configuración del proyecto frontend

Entorno: Node.js v24.12.0 Visual Studio Code y un navegador web. El frontend se ejecuta en http://localhost:5173 y la API en su propio puerto.

npm create vite@latest visualsync-web -- --template react-ts

cd visualsync-web

npm install

npm install react-router-dom axios react-hook-form zod @hookform/resolvers bootstrap

npm run dev

Herramienta	**Sugerida en la guía**	**Utilizada**
React	Sí	React con Vite
TypeScript	Sí	TypeScript
HTML5 y CSS3	Sí	HTML5, CSS3 y Bootstrap 5
Visual Studio Code	Sí	Visual Studio Code
Git y GitHub	Sí	Git y GitHub
Figma	Sí	Prototipado de las interfaces del frontend

Variables de entorno. La dirección de la API se define en el archivo .env, que no se sube al repositorio. En su lugar se incluye un .env.example:

```
VITE_API_URL=https://localhost:[PUERTO]/api
```

2. Estructura de componentes

Carpeta dentro de src/	Archivos creados
api/	client.ts, auth.ts, paquetes.ts, reservas.ts, archivos.ts
components/	Navbar.tsx, PaqueteCard.tsx, EstadoPagoBadge.tsx, ProtectedRoute.tsx, ErrorAlert.tsx, Loading.tsx, ArchivoInput.tsx
context/	AuthContext.tsx
pages/	CatalogoPage.tsx, LoginPage.tsx, RegistroPage.tsx, SolicitarReservaPage.tsx, MisReservasPage.tsx, GaleriaPage.tsx
pages/admin/	DashboardPage.tsx, PaquetesPage.tsx, ReservaDetallePage.tsx
(Raíz de src)	App.tsx, main.tsx

Carpeta	Responsabilidad
api/	Todas las llamadas HTTP. Las páginas no usan Axios directamente
components/	Piezas reutilizables: tarjetas, insignias de estado, alertas, ruta protegida

context/	Estado global de la sesión
pages/	Una pantalla por ruta, que compone los componentes
schemas/	Esquemas de validación con Zod
types/	Interfaces de TypeScript que reflejan los DTO de la API
utils/	Funciones auxiliares, como la traducción de errores a mensajes

3. Interfaces principales

Se implementan las vistas definidas en la Actividad 1 y la pantalla de administración de paquetes.

Vista	**Ruta**	**Acceso**	**Historia**	**Funcionalidad**
Catálogo de servicios	/	Público	HU-01	Tarjetas de paquetes con precio, duración, fotos y botón Reservar
Inicio de sesión	/login	Público	HU-07	Formulario con validación y mensajes de error
Registro	/registro	Público	HU-07	Creación de cuenta de Cliente

Solicitud de reserva y pago	/reservar/:idPaquete	Cliente	HU-02, HU-03	Fecha, lugar, QR de pago y carga del comprobante
Mis reservas	/mis-reservas	Cliente	HU-03	Estado de reserva y pago, nuevo comprobante si fue rechazado
Galería privada	/reservas/:id/galeria	Cliente	HU-06	Miniaturas y descarga solo con pago aprobado
Panel de administración	/admin	Admin	HU-04	Reservas, comprobantes y validación de pagos
Detalle de reserva	/admin/reservas/:id	Admin	HU-04, HU-05	Aprobar o rechazar el pago y subir fotografías
Administración de paquetes	/admin/paquetes	Admin	HU-09	CRUD de paquetes

Ilustración 1:Catálogo de servicios



Ilustración 2: Registro

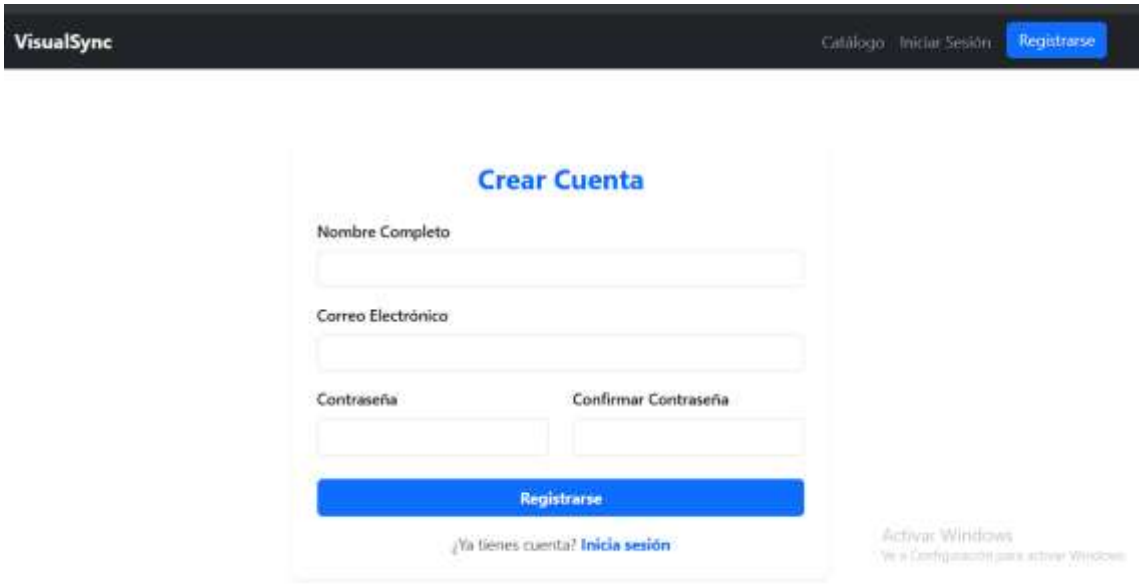


Ilustración 3: Inicio de sesión

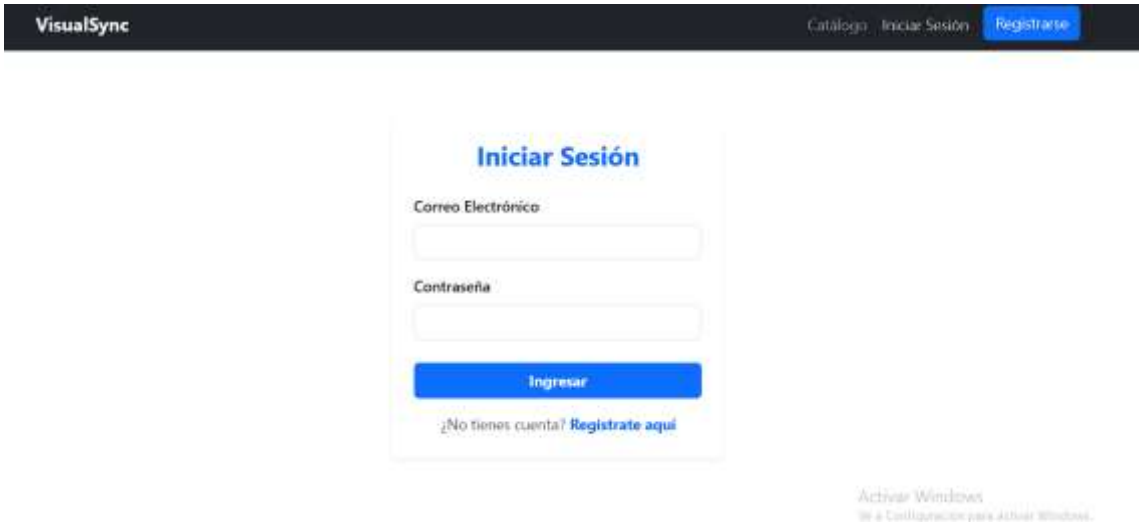


Ilustración 4: Mis reservas



4. Gestión de estados

Se usan los hooks de React y la Context API, sin librerías externas de estado, porque la aplicación no comparte datos complejos entre pantallas.

Tipo de estado	**Dónde vive**	**Herramienta**
Sesión (token y usuario)	Global	Context API (AuthContext) y localStorage
Datos del servidor (paquetes, reservas, archivos)	Local de cada página	useState y useEffect
Valores y errores de formularios	Local de cada formulario	React Hook Form
Interfaz (cargando, error, modal abierto)	Local de cada componente	useState

A continuación, se muestra el proveedor de autenticación que envuelve a la aplicación:

```
export function AuthProvider({ children }: { children: ReactNode }) {
  const [usuario, setUsuario] = useState<Usuario | null>(() => {
    const guardado = localStorage.getItem('usuario');
    return guardado ? JSON.parse(guardado) : null;
```

```
  });

  const login = (token: string, u: Usuario) => {
    localStorage.setItem('token', token);
    localStorage.setItem('usuario', JSON.stringify(u));
    setUsuario(u);
  };
  const logout = () => { localStorage.clear(); setUsuario(null); };

  return <AuthContext.Provider value={{ usuario, login, logout
}}>{children}</AuthContext.Provider>;
}
```

La sesión se restaura al recargar la página leyendo el almacenamiento local. El token expira a las 2 horas en el servidor; cuando eso ocurre, la API responde 401 y la aplicación cierra la sesión.

5. Navegación

La navegación usa React Router. Las rutas se dividen en públicas, de Cliente y de Admin. El componente ProtectedRoute redirige al login si no hay sesión y al inicio si el rol no corresponde.

```
export function ProtectedRoute({ rol, children }: { rol?: 'Admin' |
'Cliente'; children: ReactElement }) {
  const { usuario } = useAuth();
  if (!usuario) return <Navigate to='/login' replace />;
  if (rol && usuario.rol !== rol) return <Navigate to='/' replace />;
  return children;
}
```

Ruta	Componente	Protección
/	CatalogoPage	Pública
/login, /registro	LoginPage, RegistroPage	Pública
/reservar/:idPaquete	SolicitarReservaPage	Cliente

/mis-reservas	MisReservasPage	Cliente
/reservas/:id/galeria	GaleriaPage	Cliente
/admin	DashboardPage	Admin
/admin/paquetes	PaquetesPage	Admin
/admin/reservas/:id	ReservaDetallePage	Admin
*	Página no encontrada	Pública

La barra de navegación cambia según la sesión: sin sesión muestra Inicio y Login; con rol Cliente muestra Mis reservas y Cerrar sesión; con rol Admin muestra Panel, Paquetes y Cerrar sesión. Tras iniciar sesión, el Cliente va a Mis reservas y el Admin al panel.

6. Formularios

Todos los formularios usan React Hook Form, que controla los valores, los errores y el envío. Mientras se envía una petición, el botón se deshabilita para evitar envíos duplicados, y los errores devueltos por la API se muestran en una alerta sobre el formulario.

Formulario	**Campos**	**Endpoint**
Registro	Nombre completo, correo, contraseña, confirmación	POST /api/auth/registro
Inicio de sesión	Correo, contraseña	POST /api/auth/login

Solicitud de reserva	Fecha y hora, lugar	POST /api/reservas
Comprobante de pago	Archivo de imagen	POST /api/reservas/{id}/comprobante
Paquete (crear y editar)	Nombre, descripción, precio, duración, cantidad de fotos, incluye video	POST y PUT /api/paquetes
Rechazo de pago	Motivo	PATCH /api/reservas/{id}/pago
Carga de fotografías	Varios archivos de imagen	POST /api/reservas/{id}/archivos

Los archivos se envían como FormData (multipart/form-data).

7. Validación

La validación se define con Zod y se conecta a React Hook Form. Valida en el cliente antes de enviar y muestra el mensaje junto a cada campo. Esto mejora la experiencia, pero la validación definitiva sigue siendo la del backend.

```
export const paqueteSchema = z.object({
  nombre: z.string().min(3, 'El nombre es obligatorio.').max(100),
  descripcion: z.string().min(10, 'Describe el paquete.').max(500),
  precio: z.coerce.number().gt(0, 'El precio debe ser mayor a 0.'),
  duracion: z.coerce.number().int().min(1).max(24),
  cantidadFotos: z.coerce.number().int().min(1).max(5000),
  incluyeVideo: z.boolean(),
});

export const registroSchema = z.object({
  nombreCompleto: z.string().min(3, 'Ingresa tu nombre completo.'),
  email: z.string().email('Correo inválido.'),
  password: z.string().min(8, 'Mínimo 8 caracteres.'),
  confirmar: z.string(),
}).refine((d) => d.password === d.confirmar, {
```

```
  message: 'Las contraseñas no coinciden.',
  path: ['confirmar'],
});
```

8. Consumo de API REST

Toda la comunicación pasa por un cliente Axios único, con un interceptor que agrega el token JWT a cada petición y otro que reacciona a la expiración de la sesión.

```
export const client = axios.create({ baseURL:
import.meta.env.VITE_API_URL });

client.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

client.interceptors.response.use(
  (res) => res,
  (error) => {
    if (error.response?.status === 401 &&
!error.config.url?.includes('/auth/login')) {
      localStorage.clear();
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

Módulo	Funciones	Endpoints
auth.ts	registro, login	/auth/registro, /auth/login
paquetes.ts	listar, obtener, crear, actualizar, desactivar	/paquetes

reservas.ts	listar, obtener, crear, editar, cancelar, fechasOcupadas, subirComprobante, decidirPago	/reservas
archivos.ts	subirLote, listar, miniatura, descargar	/reservas/{id}/archivos, /archivos/{id}

Los tipos de TypeScript reflejan los DTO de la API, por lo que el editor avisa si se usa un campo que no existe.

9. Operaciones CRUD desde el frontend

El CRUD completo se opera desde la pantalla Administración de paquetes (rol Admin) y se complementa con las operaciones de reservas del Cliente.

Operación	Paquetes (Admin)	Reservas
Crear	Formulario de paquete POST	Solicitud de reserva POST
Leer	Lista en tabla GET	Mis reservas y panel de Admin GET
Actualizar	Formulario precargado PUT	Editar fecha y lugar (solo Solicitada) PUT
Eliminar	Botón Desactivar con confirmación DELETE	Botón Cancelar con confirmación DELETE

```
export const listar = () => client.get<Paquete[]>('/paquetes').then((r)
=> r.data);
export const crear = (d: PaqueteForm) =>
client.post<Paquete>('/paquetes', d).then((r) => r.data);
export const actualizar = (id: number, d: PaqueteForm) =>
client.put<Paquete>(`/paquetes/${id}`, d).then((r) => r.data);
```

```
export const desactivar = (id: number) =>
client.delete(`/paquetes/${id}`);
```

Después de cada operación, la lista se vuelve a consultar para reflejar el estado real del servidor. La eliminación de paquetes es una baja lógica, por lo que un paquete desactivado deja de aparecer en el catálogo.

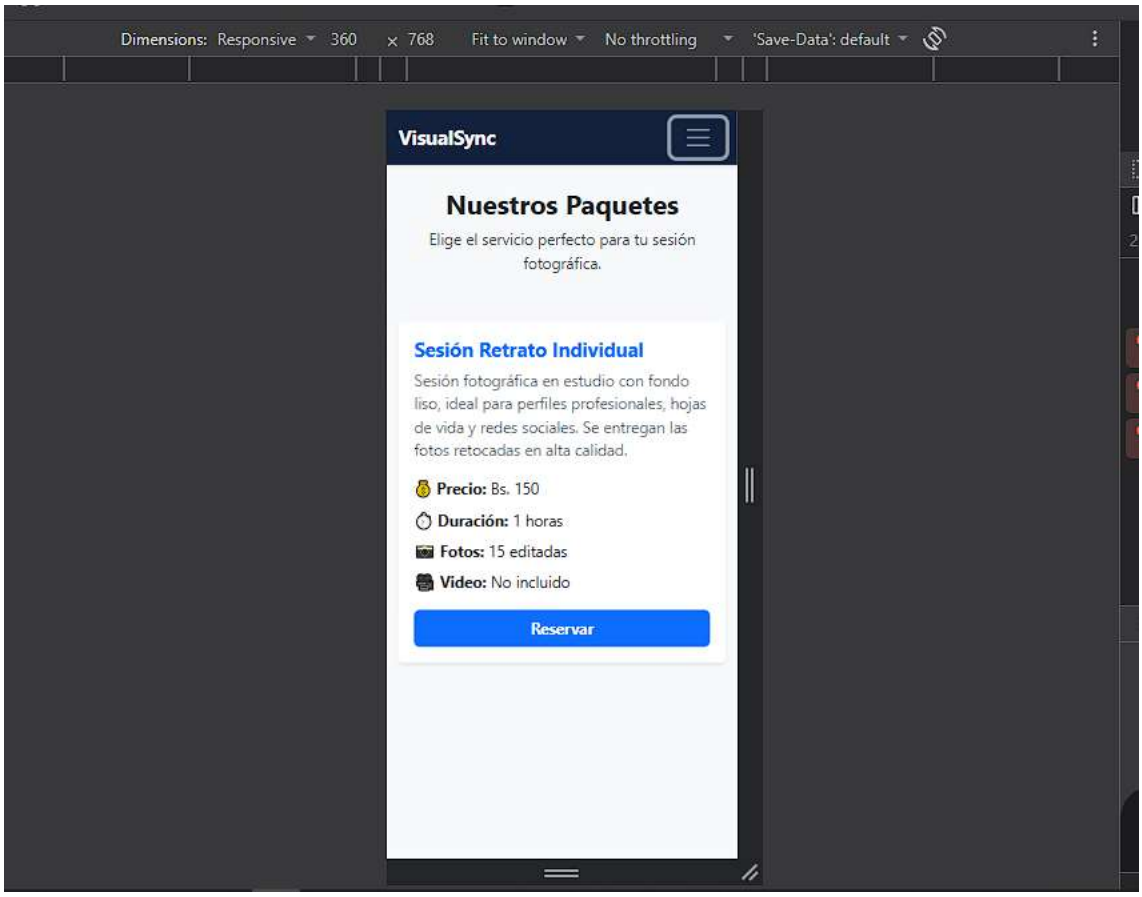
10. Diseño responsive

El diseño es mobile-first: se parte de la pantalla móvil y se amplía con los puntos de quiebre de Bootstrap 5. La página declara la etiqueta viewport y usa unidades relativas.

Elemento	**Móvil (menos de 576 px)**	**Tableta (768 px)**	**Escritorio (1200 px o más)**
Catálogo de paquetes	1 columna	2 columnas	3 columnas
Barra de navegación	Menú colapsable	Menú colapsable	Menú horizontal
Galería	2 columnas	3 columnas	4 columnas
Tablas de reservas	Desplazamiento horizontal (table-responsive)	Tabla completa	Tabla completa
Formularios	Ancho completo	Ancho limitado y centrado	Ancho limitado y centrado

Las imágenes usan img-fluid para ajustarse a su contenedor. Las pruebas de adaptación se realizaron en el modo de dispositivos del navegador con tres anchos: 360, 768 y 1280 px.

Ilustración 5: Diseño responsive



11. Integración frontend-backend

Frontend y backend son aplicaciones independientes que se comunican por HTTP y JSON. Para ello se configuró CORS en la API, que de otro modo el navegador bloquearía por tratarse de orígenes distintos.

```
builder.Services.AddCors(o => o.AddPolicy("Frontend", p =>
    p.WithOrigins("http://localhost:5173").AllowAnyHeader().AllowAnyMet
hod()));

app.UseCors("Frontend"); // antes de UseAuthentication
```

Imágenes protegidas. Las miniaturas y las descargas están protegidas por JWT, y una etiqueta de imagen no puede enviar la cabecera Authorization. Por eso el frontend pide el archivo con Axios como binario y lo muestra con una URL temporal:

```
export async function obtenerMiniatura(id: number) {
```

```
  const { data } = await client.get(`/archivos/${id}/miniatura`, {
responseType: 'blob' });
  return URL.createObjectURL(data);
}
```

La descarga de la foto original usa el mismo mecanismo y dispara el guardado del archivo. Si el pago no está aprobado, la API responde 403 y el frontend muestra el aviso, aunque el botón ya aparece bloqueado en la interfaz.

Flujo completo de punta a punta

- El visitante consulta el catálogo (GET /api/paquetes).
- Se registra e inicia sesión; el frontend guarda el token.
- Solicita la reserva (POST /api/reservas) y sube el comprobante (POST /api/reservas/{id}/comprobante).
- El Admin ve el pago pendiente y lo aprueba (PATCH /api/reservas/{id}/pago).
- El Admin sube las fotografías (POST /api/reservas/{id}/archivos).
- El Cliente abre su galería y descarga las fotos en alta resolución (GET /api/archivos/{id}/descargar).

12. Manejo de errores

Los errores de la API tienen formato uniforme (ProblemDetails), lo que permite traducirlos a mensajes para el usuario en un solo lugar.

```
export function mensajeDeError(e: unknown): string {
  if (axios.isAxiosError(e)) {
    if (!e.response) return 'No se pudo conectar con el servidor.';
    const d = e.response.data;
    if (d?.errors) return Object.values(d.errors).flat().join(' ');
    return d?.detail ?? 'Ocurrió un error inesperado.';
  }
  return 'Ocurrió un error inesperado.';
}
```

Código	**Comportamiento del frontend**
400	Muestra los mensajes de validación en el formulario

401	Cierra la sesión y redirige al login (o muestra credenciales incorrectas)
403	Muestra el aviso de que no tiene permiso
404	Muestra que el recurso no existe
409	Muestra el conflicto, por ejemplo fecha ocupada o correo repetido
500 o sin respuesta	Muestra un mensaje general y permite reintentar

Además, cada pantalla que consulta la API muestra un estado de carga, un mensaje cuando la lista está vacía y la alerta ErrorAlert cuando falla. Una ruta comodín muestra la página de no encontrado.

13. Pruebas funcionales

Las pruebas se realizaron manualmente con la aplicación y la API en ejecución.

#	Prueba	Resultado esperado	Estado	Figura
1	Ver el catálogo sin iniciar sesión	Se muestran los paquetes activos	Exitoso	Ilustración 7
2	Registro con datos inválidos	Mensajes de validación junto a los campos	Exitoso	Ilustración 8
3	Registro con correo repetido	Mensaje de conflicto (409)	Exitoso	Ilustración 9

4	Inicio de sesión con clave incorrecta	Mensaje de credenciales incorrectas	Exitoso	Ilustración 10
5	Inicio de sesión correcto	Redirige según el rol	Exitoso	Ilustración 11
6	Cliente intenta abrir `/admin`	Redirige al inicio automáticamente	Exitoso	N/A (Redirección)
7	Admin crea, edita y desactiva un paquete	La lista se actualiza en cada paso	Exitoso	Ilustración 12
8	Cliente solicita una reserva	Aparece en Mis reservas como Solicitada	Exitoso	Ilustración 13
9	Reservar una fecha ocupada	Mensaje de conflicto (409)	Exitoso	Ilustración 14
10	Subir comprobante con formato inválido	Se rechaza antes de enviar	Exitoso	Ilustración 15
11	Subir comprobante válido	El pago pasa a Pendiente de validación	Exitoso	Ilustración 15
12	Admin rechaza el pago con motivo	El cliente ve el motivo y puede subir otro	Exitoso	Ilustración 16
13	Admin aprueba el pago	La reserva pasa a Confirmada	Exitoso	Ilustración 17

14	Galería con pago pendiente	Botón de descarga bloqueado	Exitoso	Ilustración 18
15	Galería con pago aprobado	Descarga de la foto original	Exitoso	Ilustración 19
16	Token vencido	Cierra la sesión y vuelve al login	Exitoso	N/A (Redirección)
17	Vista en móvil, tableta y escritorio	El diseño se adapta en los 3 tamaños	Exitoso	Ilustración 20

14. Repositorio actualizado

- Plataforma: GitHub.
- Repositorio: https://github.com/david08234/VisualSyncMVC
- Contenido: el frontend en la carpeta frontend/ (React y TypeScript), junto al backend en VisualSync.Api/ y la documentación en docs/. Mantener ambos en el mismo repositorio permite mostrar la integración completa.
- Ramas: main (estable) y ramas feature/... por funcionalidad, fusionadas en main.
- Commits: mensajes asociados a las historias de usuario, por ejemplo feat(HU-01): agregar catálogo de paquetes, feat(HU-07): agregar login y rutas protegidas y feat(HU-06): descarga condicionada en la galería.
- Archivos ignorados: node_modules/, dist/ y .env. El archivo .env.example documenta las variables necesarias.
- README: requisitos, instalación (npm install) y ejecución (npm run dev) del frontend, y cómo levantar la API.

Ilustración 6: Commits



15.Anexos

Ilustración 7: Ver el catálogo sin iniciar sesión



Ilustración 8:Registro con datos inválidos

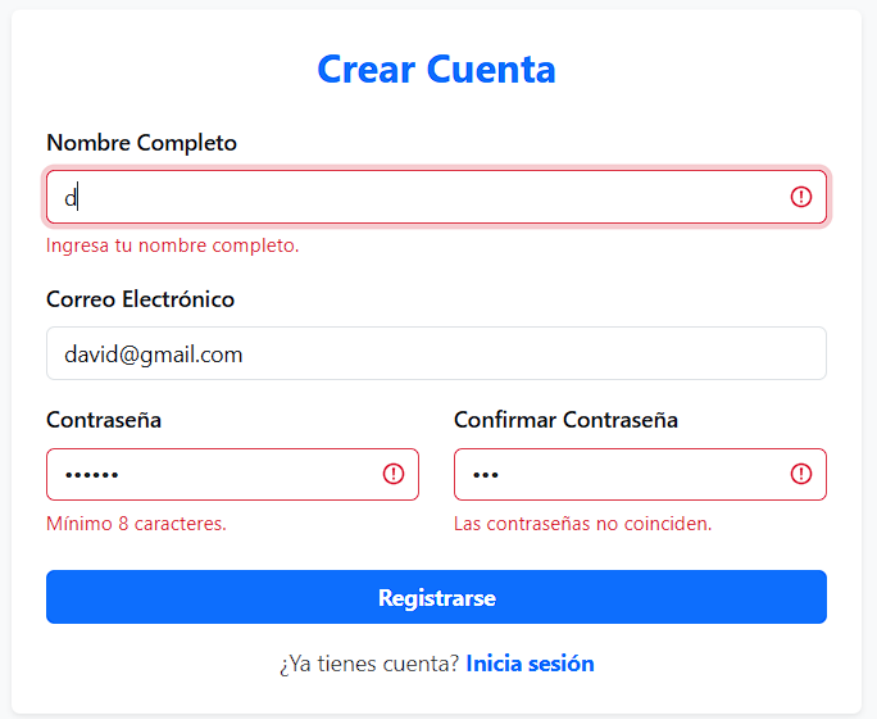


Ilustración 9: Registro con correo repetido



Ilustración 10: Inicio de sesión con clave incorrecta



Ilustración 11: Inicio de sesión correcto

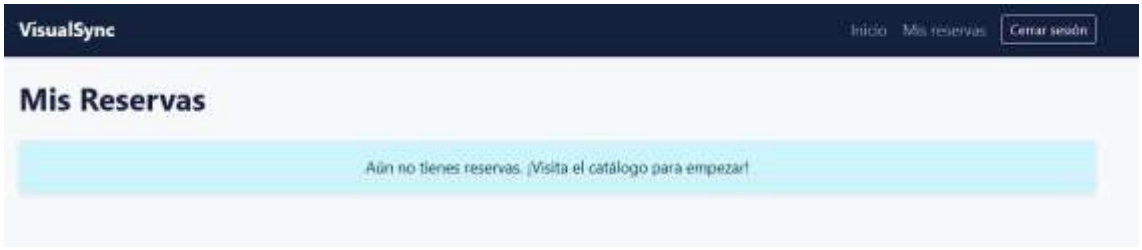


Ilustración 12: Admin crea, edita y desactiva un paquete

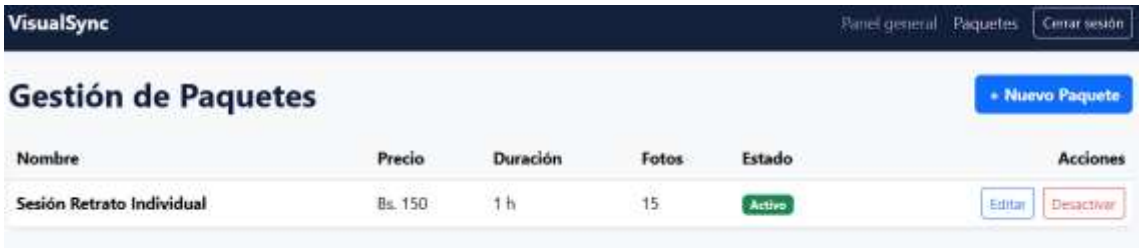


Ilustración 13: Cliente solicita una reserva



Ilustración 14: Reservar una fecha ocupada

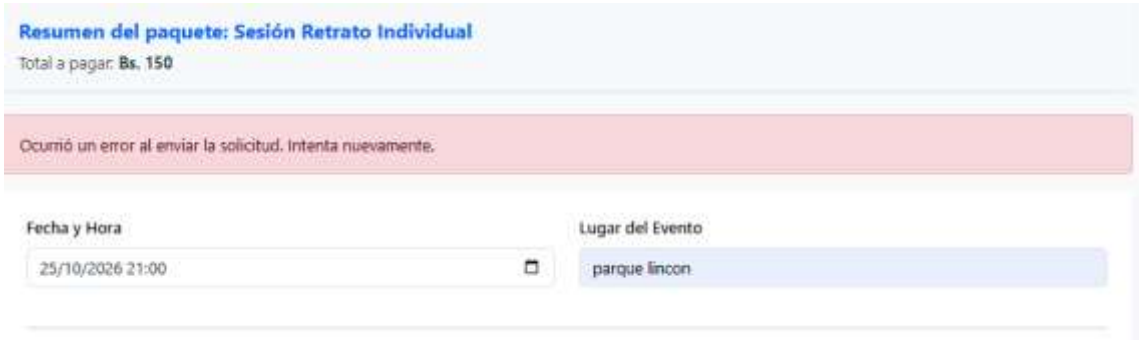


Ilustración 15: Subir comprobante válido

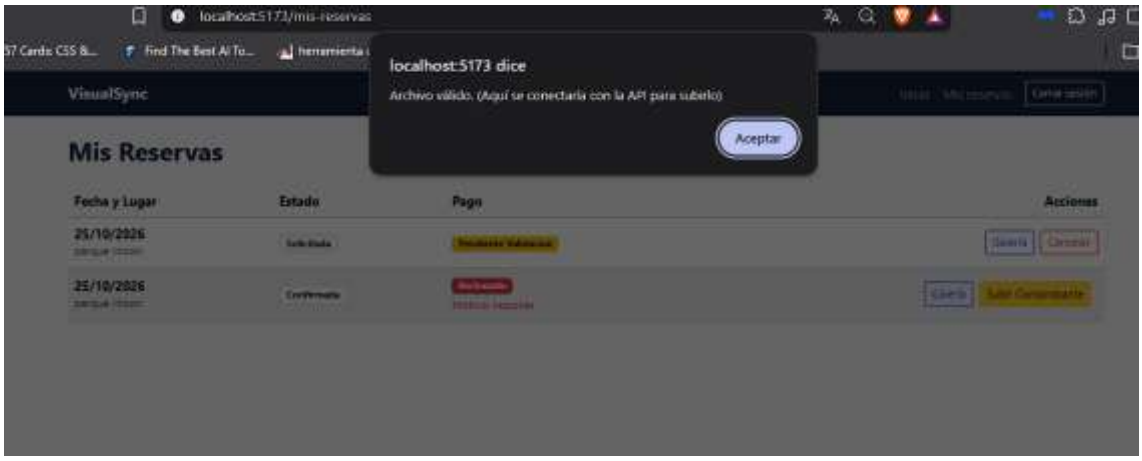


Ilustración 16: Admin rechaza el pago con motivo

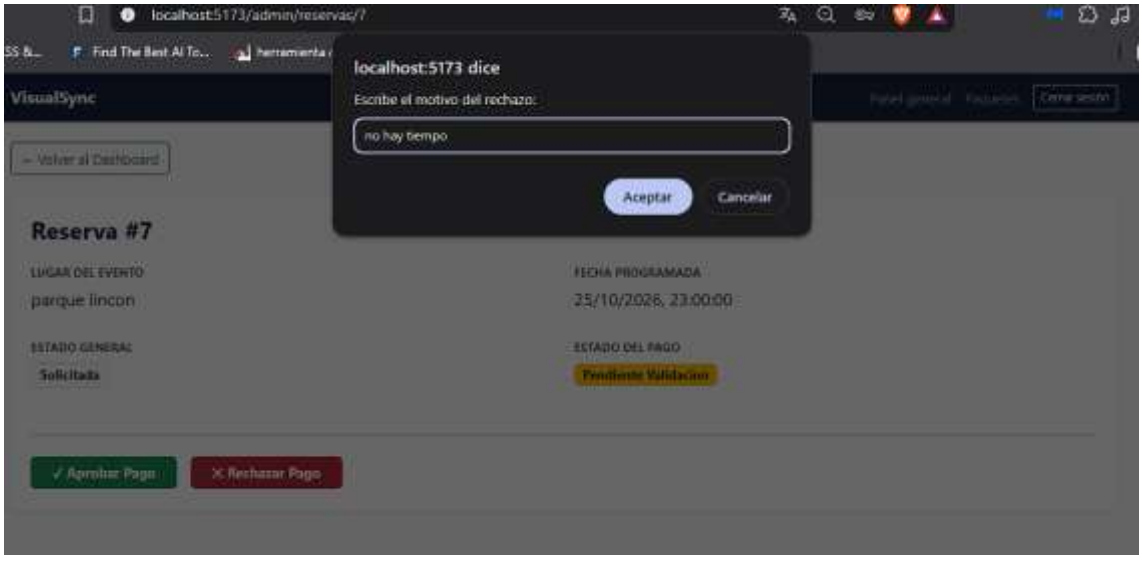


Ilustración 17: Admin aprueba el pago

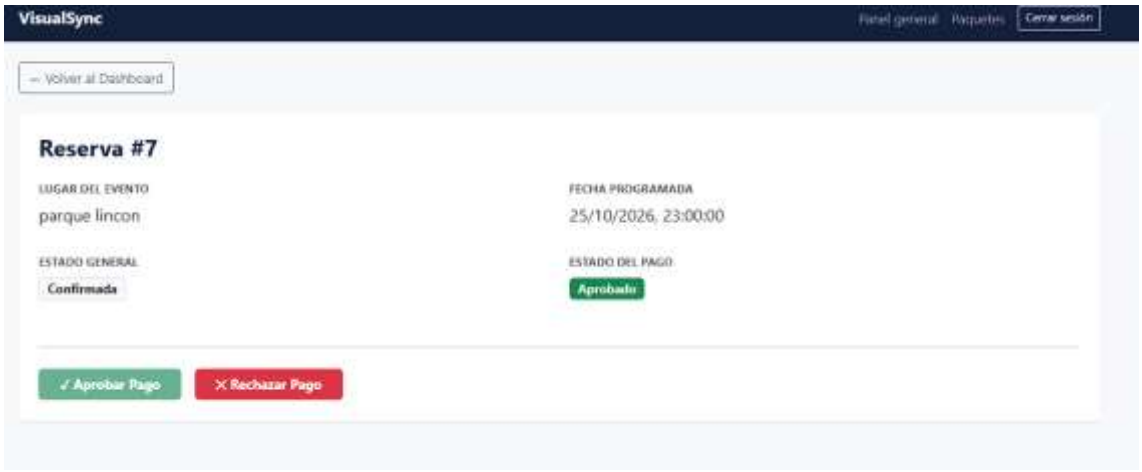


Ilustración 18: Galería con pago pendiente



Ilustración 19: Galería con pago aprobado



Ilustración 20: Vista en móvil, tableta y escritorio

